

**LAPORAN ANALISIS PERFORMA
IMPLEMENTASI PARALLEL PASSWORD HASHING
MENGUNAKAN CPU MULTITHREADING DAN GPU CUDA**



**DOSEN PENGAMPU :
SEPTIAN GEGES, S.KOM., M.KOM.**

**DISUSUN OLEH :
KELOMPOK 1**

KELVIN ANDRE HARDIAN	(2430305030011)
MUHAMMAD SHAUFI WAFA B.S	(2430205030028)
GIOVANNI MARANATHA	(2430305030022)
FAJAR OKTAVIAN RAMADHAN	(2430205030023)

**JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PALANGKA RAYA**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi komputasi modern menuntut pemrosesan data yang semakin cepat dan efisien. Salah satu pendekatan yang digunakan untuk meningkatkan performa komputasi adalah komputasi paralel, yaitu teknik yang membagi pekerjaan menjadi beberapa bagian yang dapat dieksekusi secara bersamaan oleh banyak prosesor atau inti pemrosesan.

Salah satu kasus yang memiliki kebutuhan komputasi tinggi adalah proses brute-force password hashing. Pada metode ini, sistem harus mencoba berbagai kombinasi password hingga menemukan password yang menghasilkan nilai hash yang sama dengan hash target. Seiring bertambahnya panjang password dan ukuran ruang pencarian, waktu yang dibutuhkan akan meningkat secara eksponensial.

Untuk mengatasi masalah tersebut, proyek ini mengimplementasikan dua pendekatan paralel, yaitu CPU Multithreading dan GPU Computing menggunakan CUDA melalui library Numba. Implementasi ini bertujuan untuk membandingkan performa kedua pendekatan dan menganalisis efektivitas paralelisasi menggunakan benchmark serta Amdahl's Law.

1.2 Rumusan Masalah

Rumusan masalah dalam proyek ini adalah:

- Bagaimana mengimplementasikan brute-force password hashing menggunakan CPU multithreading?
- Bagaimana memanfaatkan GPU CUDA untuk mempercepat proses komputasi?
- Bagaimana pengaruh jumlah thread terhadap performa sistem?
- Bagaimana karakteristik speedup dan efisiensi sistem paralel yang dikembangkan?

1.3 Tujuan

Tujuan proyek ini adalah:

- Mengembangkan sistem brute-force password hashing berbasis CPU dan GPU.
- Melakukan pengujian performa pada berbagai jumlah thread.
- Mengukur speedup dan efisiensi sistem.
- Menganalisis batas performa menggunakan Amdahl's Law.
- Mengevaluasi bottleneck yang muncul selama eksekusi.

BAB II

DESAIN DAN IMPLEMENTASI SISTEM

2.1 Arsitektur Sistem

Sistem terdiri atas beberapa modul utama:

- main.py
- cpu.py
- gpu.py
- hashing.py
- generator.py
- comparator.py

Alur kerja sistem:

Input Hash → Main Program → CPU / GPU Processing → Hashing → Comparator
→ Output Password

2.2 Implementasi CPU

Modul CPU menggunakan ThreadPoolExecutor untuk membagi pekerjaan ke beberapa thread.

Fitur utama:

- Sequential baseline
- Multithreading CPU
- Pengukuran waktu eksekusi
- Variasi jumlah thread

Konfigurasi yang digunakan:

- 1 Thread
- 2 Thread
- 4 Thread
- 8 Thread

2.3 Implementasi GPU

Modul GPU menggunakan CUDA melalui library Numba.

GPU digunakan untuk:

- Candidate generation
- Parallel execution

Keuntungan pendekatan GPU adalah kemampuan menjalankan ribuan thread secara bersamaan pada perangkat keras grafis.

2.4 Integrasi Sistem

Integrasi dilakukan dengan menghubungkan seluruh modul menjadi satu alur kerja lengkap.

Tahapan:

- User memasukkan hash target.
- Sistem menghasilkan kandidat password.
- Kandidat di-hash menggunakan MD5.
- Hash dibandingkan dengan target.
- Jika cocok, password ditampilkan.

BAB III

METODOLOGI PENGUJIAN

3.1 Lingkungan Pengujian

Hardware:

- Google Colab
- NVIDIA Tesla T4 GPU

Software:

- Python
- Numba
- NumPy
- Pandas
- Matplotlib
- PyTest

3.2 Skenario Benchmark

Pengujian dilakukan menggunakan berbagai konfigurasi jumlah thread (1, 2, 4, dan 8). Masing-masing konfigurasi dijalankan sebanyak lima kali dan nilai median digunakan sebagai representasi waktu eksekusi.

Thread
1
2
4
8

3.3 Metrik Pengukuran

Execution Time

Mengukur waktu yang dibutuhkan program untuk menyelesaikan proses brute-force.

Speedup

$$S(p) = T_{seq} / T_{par}$$

Efficiency

$$E(p) = S(p) / p$$

Amdahl's Law

$$S(p) = 1 / (f + (1-f)/p)$$

BAB IV

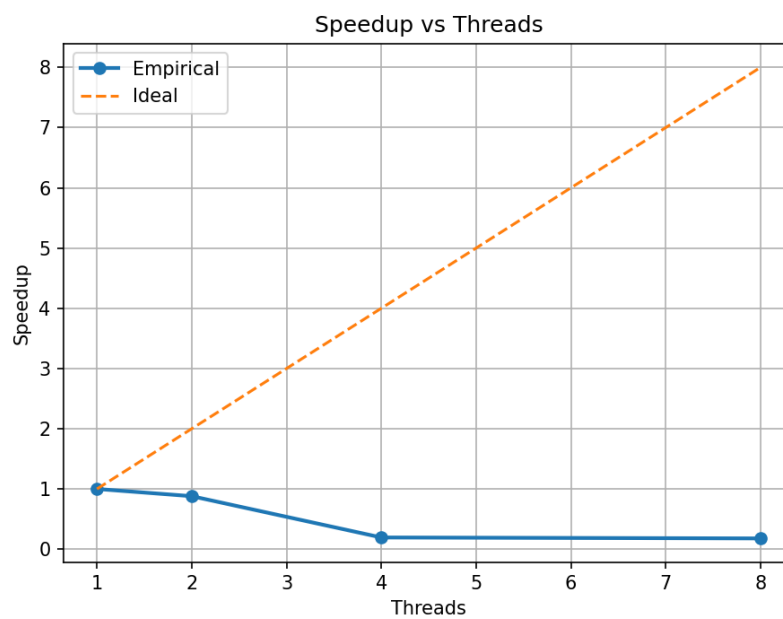
HASIL DAN ANALISIS

4.1 Hasil Benchmark

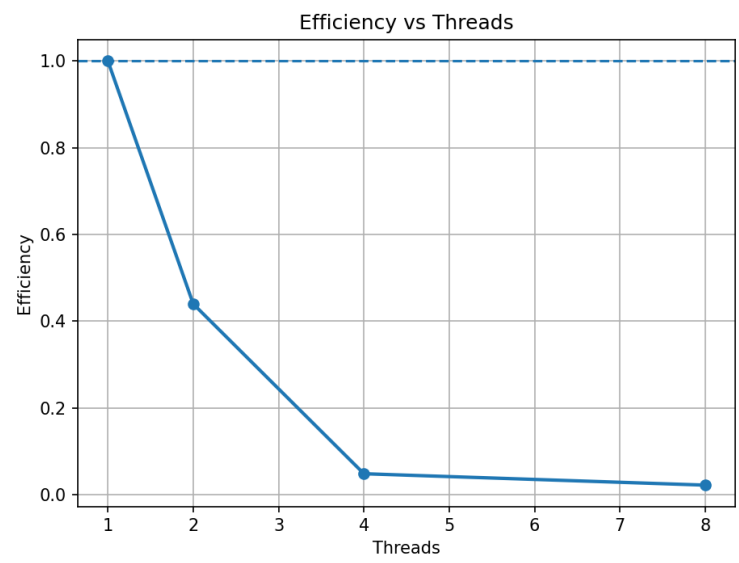
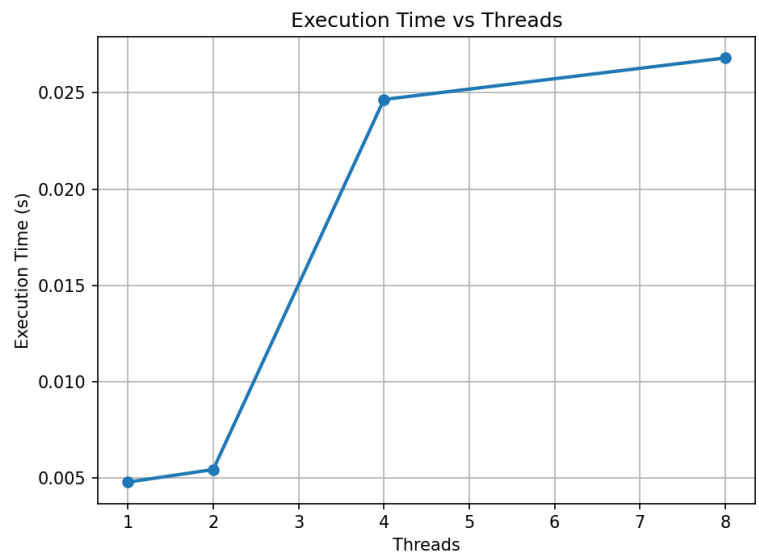
Tabel Hasil Benchmark:

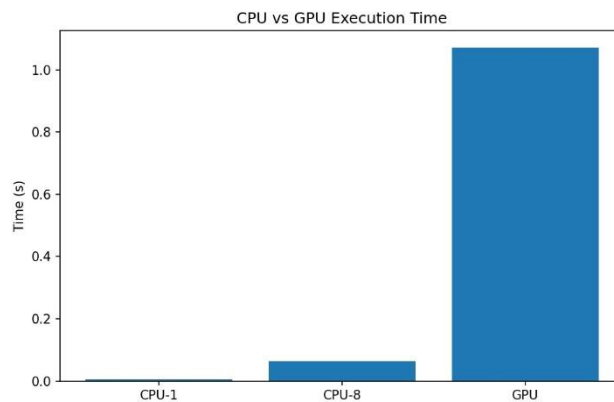
Threads	Median Time (s)	Speedup	Efficiency
1	0.004792	1.000	1.000
2	0.005446	0.880	0.440
4	0.024651	0.194	0.049
8	0.026807	0.179	0.022

4.2 Grafik Performa



Gambar 4.1 Grafik Speedup vs Thread





4.3 Analisis Speedup

Hasil eksperimen menunjukkan bahwa speedup tidak meningkat seiring bertambahnya jumlah thread. Pada 2 thread diperoleh speedup sebesar 0.88 yang lebih kecil dari 1, sehingga performa lebih lambat dibandingkan sequential baseline. Pada 4 thread speedup turun menjadi 0.194 dan pada 8 thread menjadi 0.179. Hal ini menunjukkan bahwa overhead paralelisasi lebih besar dibandingkan manfaat yang diperoleh dari pembagian pekerjaan.

4.4 Analisis Efisiensi

Efisiensi sistem menurun secara signifikan ketika jumlah thread bertambah. Rincian efisiensi adalah sebagai berikut: 1 thread = 100%, 2 thread = 44%, 4 thread = 4.9%, dan 8 thread = 2.2%. Penurunan efisiensi menunjukkan bahwa sebagian besar sumber daya digunakan untuk mengelola thread dibandingkan melakukan komputasi aktual.

4.5 Analisis Execution Time

Grafik execution time menunjukkan bahwa waktu eksekusi meningkat ketika jumlah thread bertambah. Waktu eksekusi pada 1 thread adalah 0.004792 detik dan meningkat menjadi 0.026807 detik pada 8 thread. Peningkatan ini

menunjukkan bahwa workload yang digunakan terlalu kecil sehingga overhead thread lebih dominan dibandingkan proses komputasi.

4.6 Analisis Bottleneck

Bottleneck CPU:

- Thread creation overhead
- Context switching
- Synchronization overhead
- Hash computation

Bottleneck GPU:

- Host-device transfer
- Kernel launch overhead
- Sinkronisasi CUDA
- Hashing masih dilakukan pada CPU

4.7 Analisis Amdahl's Law

Amdahl's Law menjelaskan bahwa percepatan maksimum sistem paralel dibatasi oleh bagian program yang tidak dapat diparalelkan. Hasil eksperimen menunjukkan bahwa speedup aktual selalu berada di bawah speedup ideal. Hal ini mengindikasikan bahwa overhead implementasi paralel dan bagian serial program masih mendominasi waktu eksekusi. Dengan workload yang kecil, manfaat paralelisme belum dapat dimanfaatkan secara optimal.

BAB V

PENGUJIAN DAN VALIDASI

5.1 Unit Testing

Pengujian dilakukan menggunakan PyTest.

File pengujian:

- test_hashing.py
- test_cpu.py
- test_gpu.py

Hasil: 3 passed

Seluruh modul berhasil dijalankan tanpa error.

5.2 Correctness Test

Output CPU dan GPU dibandingkan dengan sequential baseline. Hasil menunjukkan bahwa seluruh implementasi menghasilkan password yang sama terhadap hash target yang diberikan.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Proyek berhasil mengimplementasikan brute-force password hashing menggunakan CPU multithreading dan GPU CUDA. Seluruh modul berhasil diintegrasikan menjadi sistem yang berfungsi penuh mulai dari input hash hingga menghasilkan password yang sesuai.

Benchmark menunjukkan bahwa pada workload yang digunakan, peningkatan jumlah thread tidak memberikan peningkatan performa. Sequential baseline justru menghasilkan waktu eksekusi tercepat. Hasil ini menunjukkan bahwa efektivitas paralelisasi sangat bergantung pada ukuran workload dan overhead yang dihasilkan selama proses paralel.

Walaupun speedup yang diperoleh tidak optimal, proyek ini berhasil menunjukkan konsep dasar komputasi paralel, benchmarking performa, serta analisis menggunakan Amdahl's Law.

6.2 Saran

Pengembangan selanjutnya dapat dilakukan dengan:

- Memindahkan proses hashing ke GPU.
- Menambahkan dukungan SHA-256 dan SHA-512.
- Menggunakan workload yang lebih besar.
- Mengurangi overhead sinkronisasi.
- Mengimplementasikan distributed computing multi-node.
- Mengoptimalkan CUDA kernel untuk meningkatkan occupancy GPU.